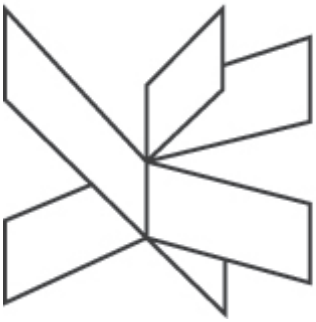
Analysis 1
Game Library Collection System

Aniara Lyn Inso Lauron: 371989
Mokhtar Buyema: 370687
Mykyta Lykholobov: 372837
Oleksandr Zavhorodnii: 370951
Sarika Jaman Sonom: 373435



VIA University College

Software Engineering, 1st Semester

Via University College
Horsens, Denmark

24th October, 2026

Contents

1 Introduction **2**
1.1 Analysis 2
1.2 Scope 2

2 Requirements **3**
2.1 Functional Requirements 3
2.2 Non-Functional Requirements 4
2.3 Use-case Diagram 4
2.4 Use-case Description 5

1 Introduction

1.1 Analysis

As players accumulate titles over time, managing a growing game collection can become increasingly difficult. This localized, single-user library management system addresses that problem by providing an isolated data environment centered on the individual gamer. The application restricts data modifications to the user's personal catalog.

The process of managing the catalog has four primary parts:

1. **Add:** The gamer can manually add a game title to their collection, including its title, genre, and description.
2. **Edit:** The gamer can update a saved game's title, genre, and description to keep the catalog up to date.
3. **Locate:** The gamer can search or filter for one or more games. Results are sorted alphabetically to make browsing easier.
4. **Remove:** The gamer can delete a game from their collection.

1.2 Scope

The system is limited to catalog management. Installing, launching, or executing games is outside its scope.

This application is a foundation for a larger system that may eventually include user connectivity. The design should allow for possible online features or community modules in the future. The current iteration focuses on making the four core catalog operations intuitive and reliable.

2 Requirements

2.1 Functional Requirements

1. **Managing games in one place.** As a gamer, I want to add games to my collection so that I can keep information about all my games in one place instead of relying on memory, notes, or different gaming platforms.
2. **Storing game information.** As a gamer, I want to store information about each game, such as its title, genre, platform, completion status, favorite status, and playtime so that I can maintain a complete overview of my collection.
3. **Finding specific games.** As a gamer, I want to search for games using different criteria, such as title, genre, or favorite status, so that I can quickly find the games I am looking for.
4. **Browsing games by genre.** As a gamer, I want to filter games by genre so that I can easily find games that match my interests and preferences.
5. **Tracking game progress.** As a gamer, I want to update a game's completion status so that I can track whether it is completed, currently in progress, or not started.
6. **Managing favorite games.** As a gamer, I want to mark games as favorites so that I can easily identify and access the games I enjoy the most.
7. **Viewing favorite games.** As a gamer, I want to view only my favorite games so that I can quickly find the games I play most often.
8. **Updating game information.** As a gamer, I want to edit game information so that outdated or incorrect information can be corrected whenever necessary.
9. **Removing games.** As a gamer, I want to remove games from my collection so that it remains organized and contains only relevant games.
10. **Recording playtime.** As a gamer, I want to record the amount of time spent playing each game so that I can monitor my gaming activity and better understand my gaming habits.
11. **Viewing playtime information.** As a gamer, I want to view the playtime of my games so that I can identify which games I spend the most time playing.
12. **Sorting games.** As a gamer, I want to sort games using different criteria so that I can organize and browse my collection more efficiently.
13. **Saving the collection.** As a gamer, I want the system to save my game collection so that my information is not lost when the application is closed.

14. **Loading saved information.** As a gamer, I want previously saved game information to load when the application starts so that I can continue using the system without entering the information again.

2.2 Non-Functional Requirements

15. **Performance and latency.** Sorting, filtering, and searching through a collection of up to 1,000 games must display results within 0.5 seconds to maintain a fast and responsive application.
16. **Usability and default behavior.** The system must sort games alphabetically by default, allowing the user to scan and evaluate their backlog at a glance.
17. **Reliability and data integrity.** The system must safely persist the user's catalog to local storage. In the event of a sudden system crash or power outage, the existing library files must not become corrupted.

2.3 Use-case Diagram

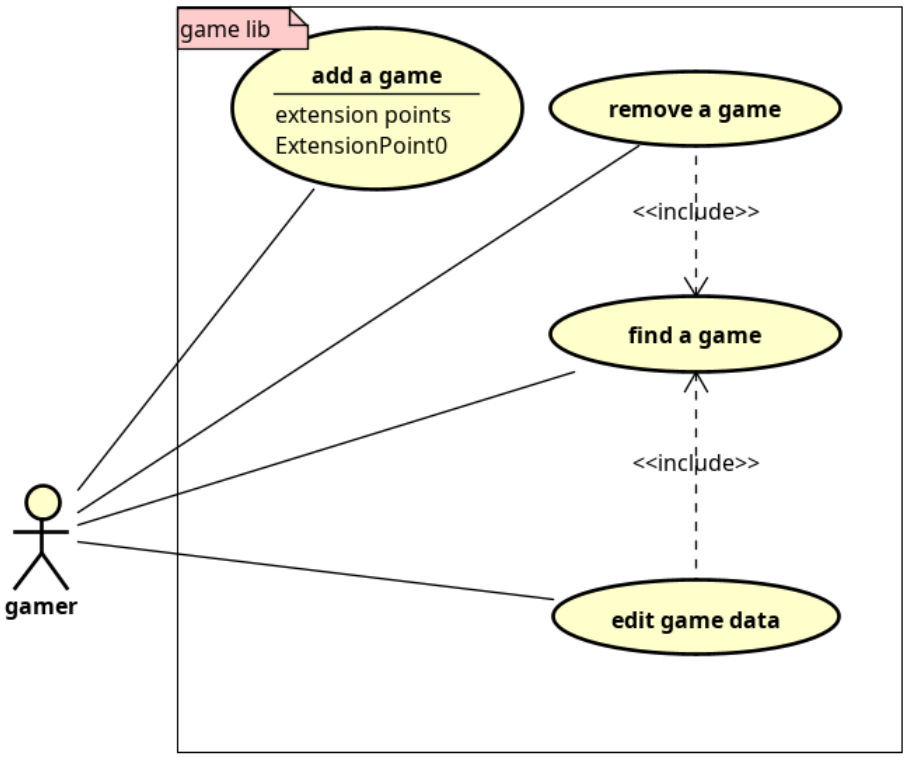


Figure 2.1: Use-case diagram for the game library collection system.

2.4 Use-case Description

Table 2.1: Use-case description: Add a game.

Use-case field	**Description**
Use case	Add a game
Summary	Allows the gamer to add a new game to their collection.
Actor	Gamer
Precondition	The game library is open and running. The gamer has access to their personal game database collection.
Postcondition	A new game has been added to the library.
Base sequence	1. The gamer selects “Add Game”. 2. The system displays a blank form. 3. The gamer enters the game details into the form. 4. The gamer selects “Save”. 5. The system validates the entered data. 6. The system saves the new game data to the database collection. 7. The system displays a success confirmation message and adds the new game to the library catalog view.
Branch sequence	*Alternative during steps 2–3 of the base sequence, while the blank form is open:* 1. The gamer closes the form or selects “Cancel”. 2. The system discards any text entered into the form fields. 3. The system returns the gamer to the main library catalog view without creating a new game entry. 4. The use case ends.
Exception sequence	*Technical crash during steps 1–3 of the base sequence:* 1. The system encounters a technical error or terminates unexpectedly before saving. 2. The system discards the uncommitted game data. 3. The postconditions cannot be realized.
Sub-use case	None

Continued on next page

Table 2.1: Use-case description: Add a game. (Continued)

Use-case field	**Description**
Note	Game data may include title, platform, genre, and release year.

Table 2.2: Use-case description: Edit game data.

Use-case field	**Description**
Use case	Edit game data
Summary	Allows the gamer to modify saved game details, such as the title and genre, to update the game catalog.
Actor	Gamer
Precondition	The gamer is part of the game library system. The game information already exists in the user's database collection.
Postcondition	The updated game information is saved to the system and displayed to the gamer.
Base sequence	1. The gamer selects "Edit Game". 2. The system displays a form with the current game details. 3. The gamer modifies the information in the form. 4. The gamer selects "Save". 5. The system validates the newly entered information. 6. The system updates the database with the new information. 7. The system displays a confirmation message and shows the updated game information.
Branch sequence	*Alternative during steps 1–3 of the base sequence:* 1. The gamer selects "Cancel". 2. The system discards the inputs. 3. The system displays the original game details without making changes. 4. The use case ends.

Continued on next page

Table 2.2: Use-case description: Edit game data. (Continued)

Use-case field	**Description**
Exception sequence	*Technical error during steps 1–5 of the base sequence:* 1. The system terminates the session or edit state. 2. The system discards the unsaved form data and newly entered details. 3. The system aborts the database transaction, leaving the original game information unchanged. 4. The postconditions cannot be realized, and the use case ends.
Sub-use case	Find a game
Note	If a technical issue closes the game library, the new information is not saved and the previous details remain in the catalog. The gamer must redo the edits to update the game information.

Table 2.3: Use-case description: Find a game.

Use-case field	**Description**
Use case	Find a game
Summary	The gamer locates one or more games in their library by searching or filtering.
Actor	Gamer
Precondition	1. The game library application is running. 2. The library contains at least one game.
Postcondition	Matching games are displayed, and the gamer can select one to view details, edit, or remove.

Continued on next page

Table 2.3: Use-case description: Find a game. (Continued)

Use-case field	**Description**
Base sequence	1. The gamer selects the “Find a game” button. 2. The system displays a search field and filter options for title, genre, and platform. 3. The gamer enters a search term and/or selects filters. 4. The system searches the library for matching games. 5. The system displays a list of matching games. 6. The gamer selects a game from the list. 7. The system displays the selected game’s details.
Branch sequence	3a. The gamer submits an empty search; the system displays all games in the library. 5a. No matching games are found; the system displays “No games found” and returns to step 3. 5b. Too many results are found; the gamer adds more filters and returns to step 3. 6a. The gamer cancels the search; the use case ends without a selection.
Exception sequence	4a. The library data cannot be loaded; the system displays an error message and the use case ends.
Sub-use case	None
Note	“Find a game” is included by “Remove a game” and “Edit game data”, since the gamer must locate a game before removing or editing it.

Table 2.4: Use-case description: Remove a game.

Use-case field	**Description**
Use case	Remove a game
Summary	Allows the gamer to remove a game from their collection.
Actor	Gamer

Continued on next page

Table 2.4: Use-case description: Remove a game. (Continued)

Use-case field	**Description**
Precondition	The gamer is logged into the game library system. The library catalog contains at least one saved game in the user's database collection.
Postcondition	The selected game record is permanently removed from the database and is no longer visible in the library catalog.
Base sequence	1. The gamer selects "Find a Game". 2. The system displays the search bar or game list. 3. The gamer selects the game they want to remove. 4. The system displays the selected game's details and a "Delete" button. 5. The gamer selects "Delete". 6. The system displays a confirmation prompt asking, "Are you sure you want to delete this game?" 7. The gamer selects "Confirm". 8. The system removes the game data from the user's database collection. 9. The system displays a success message and updates the library catalog view.
Branch sequence	*Alternative at step 6 of the base sequence, while the confirmation prompt is displayed:* 1. The gamer selects "Cancel" or "No". 2. The system closes the prompt without changing the database. 3. The system returns the gamer to the game details page. 4. The game record remains in the library catalog. 5. The use case ends.

Continued on next page

Table 2.4: Use-case description: Remove a game. (Continued)

Use-case field	**Description**
Exception sequence	*Error while the system attempts to remove the data:* 1. The system encounters a database or technical error while removing the game. 2. The system displays an error message stating that the game could not be removed. 3. The system aborts the deletion and rolls back any partial changes. 4. The game record remains in the user's collection. 5. The success postconditions cannot be realized, and the use case ends.
Sub-use case	Find a game
Note	The application asks for confirmation before removing a game.